



This project may appeal to programmers who dislike the indentation-based structure in CoffeeScript but otherwise like the language for its simplicity, terseness and expressiveness.

Although CoffeeScript was influenced by Python and Ruby, it uses neither Python's colon delimiter nor Ruby's **end** keyword. **ClosingScript** reintroduces both to clearly delimit code blocks.

Even though this adds some visual clutter, the resulting code is still way cleaner than JavaScript and has some advantages over standard CoffeeScript:

- Some people feel more comfortable with the structural integrity offered by delimited blocks.
- Whitespace is no longer significant.
- Safe copy-pasting without worrying about indentation levels.
- Safe auto-formatting.

Syntax limitations

ClosingScript is a very simple text preprocessor without a stack or a lexer. It works on line-by-line basis. As a result, it has a few syntax limitations, which are listed below. Fortunately, most of these limitations align with recommended coding practices and help improve readability.

- Keep clear the line after a trailing colon.
- No comment is allowed after a trailing arrow.
- Only use code blocks with multi-line **if**, **switch** or **try** structures.
- Unlike the other structures that are terminated with **end**, **switch** must be terminated with **end switch**.

Usage

ClosingScript outputs to *stdout*. The CoffeeScript compiler must be installed to get JavaScript.

closing <option> <script>.coffee

Options:

-c	convert code	# replace colons and end's with equivalent indentation
-f	format code	# reindent lines consistently, code is unchanged

Typical usage

```
# Convert and compile to JavaScript in one step
closing -c script.coffee | coffee -s -c > script.js
```

```
# Get a converted copy of a source
closing -c script.coffee > script.converted.coffee
```

```
# Get a formatted copy of a source
closing -f script.coffee > script.formatted.coffee
```

General syntax

```
functionName = (parms) ->  
  code  
end
```

```
functionName = (parms) -> code
```

```
do ->  
  code  
end
```

```
do -> code
```

```
[variable =] if <condition>:  
  code  
else if <condition>:  
  code  
else:  
  code  
end
```

```
[variable =] if <condition> then code [else code]
```

```
code if <condition>
```

```
[variable =] while <condition> [guard clauses]:  
  code  
end
```

```
[variable =] while <condition> [guard clauses] then code
```

```
[variable =] for <iterable> [guard clauses]:  
  code  
end
```

```
[variable =] for <iterable> [guard clauses] then code
```

```
[variable =] switch [variable]:  
  when <condition>:  
    code  
  [when <condition>:  
    code]  
  [else:  
    code]
```

```
end switch
```

the **switch** keyword is mandatory here

```
try:  
  code  
[catch [var]:  
  code]  
[finally:  
  code]  
end
```

```
class name:  
  code  
end
```